Финальный проект по React

Oct 14, 2026 · @Dias

Формат и правила

За 3 недели каждый делает full-stack приложение по одной теме из списка. Оценивается только React-фронтенд и то, как вы его объясните на защите.

- Работа индивидуальная. Тема выбирается только из списка ниже, свои темы не принимаются. Одна тема может повторяться в группе.
- Бэкенд и база данных — любые: Django/DRF, FastAPI, Node.js/Express и т.д. Бэкенд, схему БД и seed-данные можно полностью сгенерировать ИИ. Бэкенд не оценивается, но он должен быть настоящим: с JWT и реальными запросами. Только моки без сервера не принимаются.
- ИИ разрешён: Claude Code, Cursor, Copilot, ChatGPT, Gemini. Условие одно: каждую часть фронтенда вы должны уметь объяснить. Не объяснили — баллы за этот критерий не засчитываются.
- Дизайн отдельно не оценивается. Можно взять готовый шаблон из интернета, сделать макет в Claude Design или сверстать просто и аккуратно. Оценивается структура, использование библиотек и то, что всё работает.
- Всё оформление — только на Tailwind CSS. Компонентные библиотеки на Tailwind (shadcn/ui, Headless UI) разрешены. MUI, Ant Design, Bootstrap — запрещены.
- 3 часа пар в неделю на проект не хватит: рассчитывайте на самостоятельную работу дома. На парах — консультации и проверка чекпоинтов.

Темы на выбор

В каждой теме минимум 2 роли и дашборд с 3 графиками. Страницы ниже — минимум, добавлять своё можно.

№	Тема	Роли	Основные страницы	Графики на дашборде
1	Интернет-магазин электроники	Покупатель, админ	Каталог с фильтрами (категория, бренд, цена в ₸), карточка товара, корзина, оформление заказа, история заказов; админ: CRUD товаров, заказы	Выручка по дням, топ-5 товаров, заказы по статусам
2	Доставка еды	Клиент, админ ресторана	Список ресторанов, меню, корзина, заказ со статусом; админ: меню, входящие заказы	Заказы по часам, средний чек, популярные блюда
3	Аренда недвижимости	Пользователь, модератор	Лента объявлений с фильтрами (город, комнаты, цена), карточка, избранное, мои объявления; модератор: проверка объявлений	Средняя цена по городам, объявления по районам, новые объявления по дням
4	Бронирование билетов в кино	Зритель, админ	Афиша, сеансы, выбор мест на схеме зала, мои билеты; админ: фильмы и сеансы	Продажи по фильмам, заполняемость залов, выручка по дням

№	Тема	Роли	Основные страницы	Графики на дашборде
5	Запись в клинику	Пациент, врач	Врачи по специальностям, расписание, запись на слот, мои записи; врач: приёмы на день, отметка «принят»	Записи по дням недели, загрузка врачей, записи по специальностям
6	Портал вакансий	Соискатель, работодатель	Вакансии с фильтрами (город, зарплата, опыт), отклик, резюме, мои отклики; работодатель: вакансии и отклики со статусами	Воронка статусов откликов, отклики по вакансиям, зарплаты по сферам
7	Учёт личных финансов	Пользователь, админ	Транзакции, счета (карта, наличные), категории, бюджеты на месяц; админ: общие категории, пользователи	Расходы по категориям, доходы и расходы по месяцам, остаток бюджета
8	Онлайн-курсы (LMS)	Студент, преподаватель	Каталог курсов, курс с уроками, тесты, мои курсы и прогресс; преподаватель: курсы, ученики	Прогресс учеников, записи на курсы по неделям, результаты тестов

№	Тема	Роли	Основные страницы	Графики на дашборде
9	Фитнес-клуб	Клиент, админ	Абонементы, расписание групповых занятий, запись, кабинет с посещениями; админ: клиенты, абонементы, расписание	Посещаемость по дням, продажи абонементов, популярные занятия
10	Склад магазина	Сотрудник, админ	Товары и остатки, приход и расход, поставщики, список «заканчивается»; админ: сотрудники, категории	Движение товаров по дням, остатки по категориям, топ продаж

Обязательный стек фронтенда

Все библиотеки из таблицы обязательны; где указано «или» — выбор за вами.

Задача	Библиотека	Где используется
Сборка	Vite + React	Весь проект
Роутинг	React Router	Все страницы, вложенные маршруты, защищённые маршруты
Глобальное состояние	Redux Toolkit или Zustand	Авторизация, корзина/ избранное, фильтры, UI-состояние
Стили	Tailwind CSS	Вся вёрстка и адаптив; отдельные .css файлы — только index.css
Формы	React Hook Form	Все формы: логин, регистрация, создание и редактирование

Задача	Библиотека	Где используется
Валидация	Zod	Схемы для всех форм, понятные сообщения об ошибках
HTTP-клиент	Axios	Один экземпляр с baseURL из .env и interceptors для токенов
Графики	Recharts или Chart.js (react-chartjs-2)	Дашборд: минимум 3 графика разных типов
Уведомления	Sonner или react-hot-toast	Успех и ошибки запросов
Иконки	lucide-react или react-icons	Интерфейс
Даты	dayjs или date-fns	Форматирование дат и времени

Дополнительно за бонусные баллы: TypeScript, TanStack Query для загрузки данных с сервера, деплой фронтенда на Vercel или Netlify.

Обязательный функционал

Чек-лист, по которому проверяется проект. Каждая кнопка и ссылка должна работать с бэкендом — заглушки и захардкоженные массивы не засчитываются.

Авторизация (JWT)

- [ ] Регистрация и логин, сервер выдаёт access и refresh токены
- [ ] Axios interceptor добавляет заголовок Authorization: Bearer к каждому запросу
- [ ] При ответе 401 — автоматический refresh токена; если не получилось — выход и редирект на /login
- [ ] Защищённые маршруты и доступ по ролям (вторая роль видит свои страницы)
- [ ] Logout очищает токены и стор

Роутинг

- [ ] Минимум 8 маршрутов, общий layout с навигацией, вложенные маршруты
- [ ] Динамический маршрут вида /items/:id
- [ ] Страница 404
- [ ] Фильтры, поиск и страница пагинации хранятся в URL (useSearchParams)

Работа с данными

- [ ] Полный CRUD хотя бы одной сущности через интерфейс
- [ ] Поиск с debounce, фильтры, сортировка, пагинация
- [ ] Для каждого запроса есть состояния: загрузка (скелетон или спиннер), ошибка, пустой список
- [ ] Подтверждение удаления в модальном окне

Компоненты и хуки

- [ ] Понятная структура папок: pages, components, store, api, hooks
- [ ] Переиспользуемые UI-компоненты: Button, Input, Modal, Card
- [ ] Минимум 2 собственных хука (например, useDebounce, useAuth)
- [ ] React.memo, useMemo и useCallback — хотя бы по одному осмысленному применению, с объяснением зачем
- [ ] useEffect с правильными зависимостями и очисткой там, где она нужна

Дашборд

- [ ] Минимум 3 графика разных типов (линия, столбцы, круговая) на данных с бэкенда

Адаптив

- [ ] Интерфейс корректно выглядит на 375px, 768px и 1280px
- [ ] Мобильное меню (бургер), таблицы не ломают вёрстку на телефоне

План по неделям

В конце каждой недели — чекпоинт: показываете преподавателю, что работает. Чекпоинты недель 1 и 2 входят в оценку.

Неделя	Что сделать	Чекпоинт
1	Выбрать тему на первой паре. Создать репозиторий, поднять Vite + Tailwind + React Router. Сгенерировать бэкенд, БД и seed-данные. Сделать регистрацию, логин, JWT с refresh, layout и все маршруты-страницы	Логин и регистрация работают с бэкендом, защищённые маршруты закрыты, все страницы открываются

Неделя	Что сделать	Чекпоинт
2	Основные страницы с реальными данными, CRUD, стор (Redux Toolkit или Zustand), формы на React Hook Form + Zod, поиск, фильтры, пагинация	Главный сценарий темы проходит от начала до конца (например: найти товар — добавить в корзину — оформить заказ)
3	Дашборд с графиками, адаптив, состояния загрузки и ошибок, memo-оптимизации, README. Последняя пара — защита	Защита проекта

Критерии оценки и защита

Максимум 100 баллов плюс до 10 бонусных. Каждый критерий засчитывается, только если вы объяснили соответствующий код на защите.

Критерий	Баллы	Что проверяется
JWT-авторизация и роли	15	Логин, refresh в interceptor, защищённые маршруты, доступ по ролям, logout
Работа с API	15	CRUD, поиск с debounce, фильтры, пагинация, состояния загрузки, ошибки, пустого списка
Роутинг	10	Вложенные маршруты, layout, динамические параметры, 404, параметры в URL
Глобальное состояние	10	Redux Toolkit или Zustand: что лежит в сторе и почему именно там
Формы и валидация	10	React Hook Form + Zod во всех формах, ошибки показываются пользователю
Дашборд и графики	10	3 графика разных типов на реальных данных
Tailwind и адаптив	10	Только Tailwind, корректно на телефоне, планшете и десктопе
Структура и хуки	10	Папки, переиспользуемые компоненты, свои хуки, осмысленные memo / useMemo / useCallback
Чекпоинты и коммиты	10	Чекпоинты недель 1 и 2 сданы вовремя, коммиты идут все 3 недели

Критерий	Баллы	Что проверяется
Бонус	+10	TypeScript +4, TanStack Query +3, деплой +3

Как проходит защита (7–10 минут)

1. Демонстрация главного сценария под двумя ролями — 2–3 минуты.
2. Вопросы по коду: преподаватель открывает файл, вы объясняете своими словами.
3. Не смогли объяснить фрагмент — баллы за этот критерий обнуляются.
4. Проект не запускается на защите — защита переносится, минус 20 баллов.

Примеры вопросов

- Как работает обновление токена в interceptor? Что произойдёт, если refresh тоже истёк?
- Как закрыт маршрут админа? Что будет, если обычный пользователь введёт адрес вручную?
- Почему здесь useMemo или useCallback? Что изменится, если убрать?
- Почему корзина (фильтры, пользователь) лежит в сторе, а не в useState?
- Как Zod-схема связана с формой? Где показывается ошибка?
- Как данные с сервера попадают в этот график?
- Что делает этот useEffect? Почему такие зависимости?
- Как этот блок перестраивается на телефоне? Какие классы Tailwind за это отвечают?

Что сдать

Сдать до начала последней пары третьей недели:

- [ ] Ссылка на GitHub: один репозиторий с папками frontend/ и backend/ или два репозитория
- [ ] README: тема, стек, команды запуска фронта и бэка, тестовые логины для каждой роли, 3 скриншота
- [ ] .env.example без реальных секретов
- [ ] Seed-скрипт, который наполняет БД: минимум 30 записей основной сущности, чтобы графики и пагинация имели смысл
- [ ] История коммитов за все 3 недели, а не один коммит в последний день
- [ ] По желанию: ссылка на деплой